

GA – Practica en clase · Semana 5 · Unidad II · Formativa | Individual

Instalacion de entorno y primer script

PHP 8.2 y PERL/CGI — Procesamiento seguro de formularios web

Formativa

Individual

Presencial

Entrega: PDF con evidencias en el LMS

Descripcion de la practica

Esta practica introduce al estudiante en la programacion del lado del servidor mediante dos tecno-
logias fundacionales de la web: **PHP 8.2** y **PERL 5.38 con el modulo CGI.pm**. El estudiante
configurara un entorno de desarrollo local (XAMPP o Docker), construira un formulario HTML
comun y desarrollara dos scripts independientes —uno en cada lenguaje— que procesen, validen y
saneen los datos enviados por el usuario. La actividad permite comparar de forma directa la filosofia,
la sintaxis y los mecanismos de seguridad de ambos lenguajes ante el mismo problema practico: el
procesamiento seguro de entradas de usuario en aplicaciones web dinamicas.

PERL fue el primer lenguaje de scripting web de uso masivo y sigue siendo relevante en administra-
cion de sistemas, bioinformatica y automatizacion de texto. PHP domina actualmente el 77 % del
mercado de servidores con lenguaje conocido (W3Techs, 2024). Conocer ambos brinda al estudian-
te perspectiva historica y tecnica sobre la evolucion de la programacion del servidor, y le permite
identificar cual de los dos enfoques es mas adecuado para diferentes escenarios profesionales.

Conexion con el Proyecto Fin de Curso (PFC)

El backend del PFC se desarrollara en PHP 8.2 como tecnologia principal. Esta practica es-
tablece las bases del procesamiento seguro de formularios que se utilizara en el modulo de
autenticacion y en el CRUD de la Entrega 1A. El codigo escrito hoy es el punto de partida del
repositorio del equipo.

Asignatura	Aplicaciones Web
Unidad	Unidad II — Herramientas de Programacion Web del Servidor
Tema relacionado	Tema 5: Introduccion a la Programacion del Lado del Servidor
Tipo de actividad	Practica en clase (Gestion de Aula)
Caracter	Formativa (no sumativa)
Modalidad	Individual — presencial en el aula/laboratorio
Tecnologias	PHP 8.2 con Apache/XAMPP PERL 5.38 con CGI.pm
Entregable	PDF con capturas de pantalla y tabla comparativa, subido al LMS
Semana	Semana 5 — Del 08 al 14 de Diciembre del 2025

Directrices

1 Instalacion y verificacion del entorno de desarrollo

Opcion A — XAMPP (recomendada en laboratorio):

1. Descargar XAMPP 8.2.x desde <https://www.apachefriends.org/> e instalar con las opciones por defecto. Seleccionar obligatoriamente los componentes **Apache**, **PHP** y **Perl**.
2. Iniciar Apache y MySQL desde el panel de control de XAMPP.
3. Crear la carpeta de trabajo: C:\xampp\htdocs\practica5\ (Windows) o /opt/lampp/htdocs/practica5/ (Linux/Mac).
4. El directorio CGI para PERL ya existe en C:\xampp\cgi-bin\ (o /opt/lampp/cgi-bin/). Verificar que el modulo CGI de Apache esta habilitado revisando que en httpd.conf existan y no esten comentadas las lineas:

Terminal — Verificar configuracion Apache para CGI

```
# En httpd.conf (buscar con Ctrl+F estas lineas):
LoadModule cgi_module modules/mod_cgi.so
# Tambien debe existir el bloque para cgi-bin:
ScriptAlias /cgi-bin/ "C:/xampp/cgi-bin/"
```

Opcion B — Docker (entorno reproducible):

Terminal — Docker Compose con PHP y PERL

```
# docker-compose.yml
version: '3.8'
services:
  php:
    image: php:8.2-apache
    ports: ["8080:80"]
    volumes: ["/php-app:/var/www/html"]
    command: >
      sh -c "a2enmod cgi && apache2-foreground"

  perl:
    image: perl:5.38
    ports: ["8081:8080"]
    volumes: ["/perl-app:/app"]
    command: >
      sh -c "cpan CGI &&
        cd /app &&
        perl -MCGI::Util -e '1' &&
        plackup -p 8080 app.psgi"
```

Verificacion de la instalacion:

Terminal — Comprobar versiones instaladas

```
# Verificar PHP
php -v
# Resultado esperado: PHP 8.2.x (cli) ...

# Verificar Perl
perl -v
# Resultado esperado: This is perl 5, version 38 ...

# Verificar modulo CGI de Perl
perl -MCGI -e 'print "CGI.pm OK\n"'
# Resultado esperado: CGI.pm OK

# Verificar que Apache responde
curl http://localhost/
# Resultado esperado: pagina de bienvenida de XAMPP
```

Estructura de archivos a crear

htdocs/practica5/formulario.html	Formulario HTML comun
htdocs/practica5/procesar.php	Script de procesamiento en PHP
cgi-bin/procesar.pl	Script de procesamiento en PERL

Los tres archivos forman una unidad: el mismo formulario envia datos a ambos scripts.

2**HTML5****Formulario HTML comun para PHP y PERL**

Crear `formulario.html`. El formulario tiene **dos botones de envio**: uno apunta a `procesar.php` y otro a `/cgi-bin/procesar.pl`, permitiendo probar ambos backends desde la misma interfaz.

HTML — formulario.html — Formulario compartido PHP y PERL

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Practica 5 - PHP y PERL/CGI</title>
7   <style>
8     body { font-family: Arial, sans-serif; max-width: 600px;
9           margin: 2rem auto; padding: 0 1rem; }
10    .form-group { margin-bottom: 1rem; }
11    label { display: block; font-weight: bold; margin-bottom: 4px; }
12    input, textarea { width: 100%; padding: 8px; border: 1px solid #ccc;
13                   border-radius: 4px; box-sizing: border-box; }
14    .btn-group { display: flex; gap: 12px; margin-top: 1rem; }
15    button { flex: 1; padding: 10px; border: none; border-radius: 4px;
16           cursor: pointer; font-size: 1rem; color: white; }
17    .btn-php { background: #7779b3; }
18    .btn-perl { background: #3d8b3d; }
19    button:hover { opacity: 0.88; }
```

```
20     </style>
21 </head>
22 <body>
23     <h1>Formulario de registro</h1>
24     <p>Completa todos los campos. Prueba con PHP o con PERL/CGI.</p>
25
26     <form id="formulario" method="POST">
27         <div class="form-group">
28             <label for="nombre">Nombre completo *</label>
29             <input type="text" id="nombre" name="nombre"
30                 placeholder="Ej: Maria Garcia" maxlength="100">
31         </div>
32
33         <div class="form-group">
34             <label for="email">Correo electronico *</label>
35             <input type="email" id="email" name="email"
36                 placeholder="usuario@dominio.com">
37         </div>
38
39         <div class="form-group">
40             <label for="edad">Edad (1-120) *</label>
41             <input type="number" id="edad" name="edad"
42                 min="1" max="120" placeholder="Ej: 22">
43         </div>
44
45         <div class="form-group">
46             <label for="mensaje">Mensaje *</label>
47             <textarea id="mensaje" name="mensaje" rows="4"
48                 maxlength="500" placeholder="Escribe tu mensaje aqui..."
49                 ></textarea>
50         </div>
51
52         <div class="btn-group">
53             <button type="submit" class="btn-php"
54                 formaction="procesar.php">
55                 Enviar con PHP
56             </button>
57             <button type="submit" class="btn-perl"
58                 formaction="/cgi-bin/procesar.pl">
59                 Enviar con PERL/CGI
60             </button>
61         </div>
62     </form>
63
64     <hr>
65     <p style="font-size:0.85rem; color:#666;">
66         Practica 5 - Aplicaciones Web - UTEQ 2025-2026
67     </p>
68 </body>
69 </html>
```

Por que un formulario comun

Usar un unico `formulario.html` para ambos scripts permite comparar de forma directa el comportamiento de PHP y PERL ante exactamente los mismos datos de entrada. El atributo `formaction` en cada boton redirige el envio al backend correspondiente sin modificar el formulario.

3

PHP 8.2

Script PHP 8.2 con validacion y saneamiento

Crear `procesar.php` en `htdocs/practica5/`. El script debe:

- Verificar que el metodo de envio es POST (**nunca procesar datos sin verificar el metodo**).
- Usar `filter_input()` para leer y validar/sanear cada campo.
- Usar `htmlspecialchars()` en toda salida para prevenir XSS.
- Mostrar un resumen de los datos validos o los mensajes de error especificos.
- Incluir un enlace para volver al formulario.

PHP — procesar.php — Validacion y saneamiento seguro

```
1 <?php
2 declare(strict_types=1);
3
4 // 1. Verificar metodo HTTP
5 if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
6     header('Location: formulario.html');
7     exit;
8 }
9
10 // 2. Leer y validar cada campo con filter_input()
11 $errores = [];
12
13 // Nombre: sanear y verificar longitud
14 $nombre = filter_input(INPUT_POST, 'nombre',
15                         FILTER_DEFAULT);
16 $nombre = htmlspecialchars(trim($nombre ?? ''),
17                             ENT_QUOTES, 'UTF-8');
18 if (empty($nombre)) {
19     $errores['nombre'] = 'El nombre no puede estar vacio.';
20 } elseif (strlen($nombre) < 2 || strlen($nombre) > 100) {
21     $errores['nombre'] =
22         'El nombre debe tener entre 2 y 100 caracteres.';
23 }
24
25 // Email: validar formato con FILTER_VALIDATE_EMAIL
26 $email = filter_input(INPUT_POST, 'email',
27                       FILTER_VALIDATE_EMAIL);
28 if ($email === false || $email === null) {
29     $errores['email'] =
30         'El correo electronico no tiene un formato valido.';
```

```
31 } else {
32     $email = htmlspecialchars($email, ENT_QUOTES, 'UTF-8');
33 }
34
35 // Edad: validar entero en rango 1-120
36 $opcionesEdad = [
37     'options' => ['min_range' => 1, 'max_range' => 120]
38 ];
39 $edad = filter_input(INPUT_POST, 'edad',
40                     FILTER_VALIDATE_INT, $opcionesEdad);
41 if ($edad === false || $edad === null) {
42     $errores['edad'] =
43         'La edad debe ser un numero entero entre 1 y 120.';
44 }
45
46 // Mensaje: sanear y verificar longitud
47 $mensaje = filter_input(INPUT_POST, 'mensaje',
48                         FILTER_DEFAULT);
49 $mensaje = htmlspecialchars(trim($mensaje ?? ''),
50                             ENT_QUOTES, 'UTF-8');
51 if (empty($mensaje)) {
52     $errores['mensaje'] = 'El mensaje no puede estar vacio.';
53 } elseif (strlen($mensaje) > 500) {
54     $errores['mensaje'] =
55         'El mensaje no puede superar los 500 caracteres.';
56 }
57
58 // 3. Mostrar resultado
59 ?>
60 <!DOCTYPE html>
61 <html lang="es">
62 <head>
63     <meta charset="UTF-8">
64     <title>Resultado PHP</title>
65     <style>
66         body { font-family: Arial, sans-serif; max-width: 600px;
67             margin: 2rem auto; padding: 0 1rem; }
68         .error { color: #c0392b; background: #fdecea;
69             padding: 8px 12px; border-radius: 4px;
70             border-left: 4px solid #c0392b; margin: 4px 0; }
71         .exito { color: #1e6b3c; background: #eaf5ee;
72             padding: 8px 12px; border-radius: 4px;
73             border-left: 4px solid #1e6b3c; margin: 4px 0; }
74         table { width: 100%; border-collapse: collapse; }
75         td, th { padding: 8px 12px; border: 1px solid #ddd;
76             text-align: left; }
77         th { background: #7779b3; color: white; }
78         tr:nth-child(even) { background: #f5f5f5; }
79         .badge { display: inline-block; padding: 2px 10px;
80             border-radius: 12px; font-size: 0.8rem;
81             background: #7779b3; color: white; }
82     </style>
83 </head>
```

```
84 <body>
85   <h1><span class="badge">PHP 8.2</span>
86     Resultado del procesamiento</h1>
87
88   <?php if (!empty($errores)): ?>
89     <h2 style="color:#c0392b;">Se encontraron errores:</h2>
90     <?php foreach ($errores as $campo => $msg): ?>
91       <div class="error">
92         <strong><?= htmlspecialchars(ucfirst($campo),
93           ENT_QUOTES, 'UTF-8') ?>:</strong>
94         <?= $msg ?>
95       </div>
96     <?php endforeach; ?>
97
98   <?php else: ?>
99     <div class="exito">
100       <strong>Datos recibidos y validados correctamente.</strong>
101     </div>
102     <h2>Resumen de los datos:</h2>
103     <table>
104       <tr><th>Campo</th><th>Valor recibido</th></tr>
105       <tr><td>Nombre</td><td><?= $nombre ?></td></tr>
106       <tr><td>Email</td><td><?= $email ?></td></tr>
107       <tr><td>Edad</td><td><?= $edad ?> años</td></tr>
108       <tr><td>Mensaje</td><td><?= $mensaje ?></td></tr>
109     </table>
110   <?php endif; ?>
111
112   <p><a href="formulario.html">&larr; Volver al formulario</a></p>
113   <hr>
114   <p style="font-size:0.8rem;color:#888;">
115     Procesado por PHP <?= phpversion() ?> |
116     Metodo: POST |
117     IP: <?= htmlspecialchars(
118       $_SERVER['REMOTE_ADDR'] ?? 'desconocida',
119       ENT_QUOTES, 'UTF-8') ?>
120   </p>
121 </body>
122 </html>
```

Funciones clave de PHP usadas

<code>filter_input()</code>	Lee una variable de entrada (<code>INPUT_POST</code>) aplicando un filtro de validación o saneamiento. Mas seguro que acceder directamente a <code>\$_POST</code> .
<code>FILTER_VALIDATE_EMAIL</code>	Valida que la cadena tenga formato de email segun RFC 5321. Devuelve <code>false</code> si no es valido.
<code>FILTER_VALIDATE_INT</code>	Valida que el valor sea un entero; con <code>options</code> permite definir rango minimo y maximo.
<code>htmlspecialchars()</code>	Convierte caracteres especiales HTML (<, >, ", &) en entidades HTML, previniendo ataques XSS.
<code>declare(strict_types=1)</code>	Activa el modo de tipos estrictos de PHP 8: las funciones que esperan <code>int</code> rechazaran una cadena.

4**PERL/CGI****Script PERL/CGI con validacion y saneamiento**

Crear `procesar.pl` en el directorio `cgi-bin/`. Consideraciones previas obligatorias:

1. **La primera linea** del script (*shebang*) debe indicar la ruta al interprete Perl: `#!/usr/bin/perl` (Linux/Mac) o `#!/usr/local/bin/perl` (XAMPP Windows).
2. **Permisos de ejecucion:** en Linux/Mac ejecutar `chmod 755 procesar.pl`. En Windows con XAMPP, Apache ya tiene permiso de ejecucion en el `cgi-bin`.
3. **Pragma obligatorio:** siempre usar `use strict;` y `use warnings;` — es la forma correcta y profesional de escribir Perl moderno.
4. **Cabecera HTTP:** la primera salida del script DEBE ser la cabecera HTTP; de lo contrario Apache genera un error 500.

PERL/CGI — procesar.pl — Validacion y saneamiento en PERL/CGI

```

1  #!/usr/bin/perl
2  use strict;
3  use warnings;
4  use CGI;
5  use CGI::Carp qw(fatalsToBrowser); # Muestra errores en browser
6
7  # 1. Crear objeto CGI y emitir cabecera HTTP PRIMERO
8  my $cgi = CGI->new;
9  print $cgi->header(-type => 'text/html', -charset => 'UTF-8');
10
11 # 2. Leer parametros del formulario (metodo POST)
12 my $nombre = $cgi->param('nombre') // '';
13 my $email = $cgi->param('email') // '';
14 my $edad = $cgi->param('edad') // '';
15 my $mensaje = $cgi->param('mensaje') // '';
16
17 # 3. Verificar metodo HTTP
18 my $metodo = $cgi->request_method() // '';
19 if ($metodo ne 'POST') {

```



```
20     print $cgi->redirect('formulario.html');
21     exit;
22 }
23
24 # 4. Funcion de saneamiento: equivalente a htmlspecialchars de PHP
25 sub sanear {
26     my ($texto) = @_ ;
27     $texto = CGI::escapeHTML($texto); # Escapa <>&"'
28     $texto =~ s/\s+|\s+$//g;          # trim (quitar espacios)
29     return $texto;
30 }
31
32 # 5. Validaciones
33 my %errores;
34
35 # Validar nombre
36 $nombre = sanear($nombre);
37 if (length($nombre) < 2) {
38     $errores{nombre} = 'El nombre no puede estar vacio.';
39 } elsif (length($nombre) > 100) {
40     $errores{nombre} =
41         'El nombre no puede superar los 100 caracteres.';
42 }
43
44 # Validar email con expresion regular (RFC 5321 simplificado)
45 $email = sanear($email);
46 unless ($email =~
47     /^[a-zA-Z0-9._%+\-]+\@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$/) {
48     $errores{email} =
49         'El correo electronico no tiene formato valido.';
50 }
51
52 # Validar edad: debe ser entero en rango 1-120
53 $edad = sanear($edad);
54 unless ($edad =~ /\d+$/ && $edad >= 1 && $edad <= 120) {
55     $errores{edad} =
56         'La edad debe ser un numero entero entre 1 y 120.';
57 }
58
59 # Validar mensaje
60 $mensaje = sanear($mensaje);
61 if (length($mensaje) == 0) {
62     $errores{mensaje} = 'El mensaje no puede estar vacio.';
63 } elsif (length($mensaje) > 500) {
64     $errores{mensaje} =
65         'El mensaje no puede superar los 500 caracteres.';
66 }
67
68 # 6. Generar respuesta HTML
69 my $version_perl = sprintf("%vd", $^V);
70
71 print <<HTML_INICIO;
72 <!DOCTYPE html>
```

```

73 <html lang="es">
74 <head>
75   <meta charset="UTF-8">
76   <title>Resultado PERL/CGI</title>
77   <style>
78     body { font-family: Arial, sans-serif; max-width: 600px;
79           margin: 2rem auto; padding: 0 1rem; }
80     .error { color: #c0392b; background: #fdecea;
81            padding: 8px 12px; border-radius: 4px;
82            border-left: 4px solid #c0392b; margin: 4px 0; }
83     .exito { color: #1e6b3c; background: #eaf5ee;
84            padding: 8px 12px; border-radius: 4px;
85            border-left: 4px solid #1e6b3c; }
86     table { width:100%; border-collapse:collapse; }
87     td, th { padding:8px 12px; border:1px solid #ddd; }
88     th     { background:#3d8b3d; color:white; }
89     tr:nth-child(even) { background:#f5f5f5; }
90     .badge { display:inline-block; padding:2px 10px;
91            border-radius:12px; font-size:0.8rem;
92            background:#3d8b3d; color:white; }
93   </style>
94 </head>
95 <body>
96   <h1><span class="badge">PERL/CGI</span>
97     Resultado del procesamiento</h1>
98   HTML_INICIO
99
100  if (%errores) {
101    print "<h2 style='color:#c0392b;'>Se encontraron errores:</h2>\n";
102    foreach my $campo (sort keys %errores) {
103      my $campo_display = ucfirst($campo);
104      print "<div class='error'><strong>$campo_display:</strong> "
105            . $errores{$campo} . "</div>\n";
106    }
107  } else {
108    print "<div class='exito'>";
109    print "<strong>Datos recibidos y validados correctamente.</strong>";
110    print "</div>\n";
111    print "<h2>Resumen de los datos:</h2>\n";
112    print "<table>\n";
113    print "<tr><th>Campo</th><th>Valor recibido</th></tr>\n";
114    print "<tr><td>Nombre</td><td>$nombre</td></tr>\n";
115    print "<tr><td>Email</td><td>$email</td></tr>\n";
116    print "<tr><td>Edad</td><td>$edad anos</td></tr>\n";
117    print "<tr><td>Mensaje</td><td>$mensaje</td></tr>\n";
118    print "</table>\n";
119  }
120
121  print <<HTML_FIN;
122    <p><a href="/practica5/formulario.html">&larr; Volver</a></p>
123    <hr>
124    <p style="font-size:0.8rem;color:#888;">
125    Procesado por Perl $version_perl con CGI.pm |

```

```

126     Metodo: POST
127     </p>
128 </body>
129 </html>
130 HTML_FIN

```

Elementos clave de PERL/CGI usados

<code>use strict;</code>	Obliga a declarar todas las variables con <code>my</code> . Evita errores por nombres de variables mal escritos. Equivalente al modo estricto de JS.
<code>use warnings;</code>	Activa advertencias de Perl en tiempo de ejecución. Complemento obligatorio de <code>strict</code> .
<code>CGI->new</code>	Crea un objeto CGI que encapsula la petición HTTP (equivalente a <code>\$_POST</code> en PHP, pero orientado a objetos).
<code>\$cgi->param()</code>	Lee el valor de un parámetro del formulario. Equivalente a <code>filter_input(INPUT_POST, ...)</code> en PHP.
<code>CGI::escapeHTML()</code>	Escapa caracteres HTML especiales. Equivalente exacto de <code>htmlspecialchars()</code> en PHP.
<code>// (defined-or)</code>	Operador de Perl 5.10+: si el lado izquierdo es <code>undef</code> , usa el valor de la derecha (equivalente al operador <code>??</code> de PHP 8).
Heredoc	Permite escribir bloques de HTML multilínea sin concatenar cadenas. Más legible que múltiples <code>print</code> .
<code>\$^V</code>	Variable especial de Perl: versión del intérprete.

5 Verificación con casos de prueba

Probar los siguientes casos en **ambos** scripts y documentar la respuesta obtenida en el informe. Cada caso debe probarse de forma independiente enviando el formulario.

#	Caso de prueba	Datos de entrada	Resultado esperado	Aplica a
1	Campos vacíos	Formulario sin rellenar (enviar vacío)	Error en los 4 campos: nombre, email, edad y mensaje	PHP y PERL
2	Email sin dominio	nombre=Ana, email=usuario@, edad=20, msg=Hola	Error solo en email: formato inválido	PHP y PERL
3	Email sin arroba	email=usuariodominio.com	Error solo en email	PHP y PERL
4	Edad negativa	edad=-5	Error solo en edad: fuera del rango 1-120	PHP y PERL
5	Edad demasiado alta	edad=200	Error solo en edad: fuera del rango 1-120	PHP y PERL
6	Edad no numérica	edad=veinte	Error solo en edad: debe ser entero	PHP y PERL
7	Intento XSS	nombre=<script>alert(1)</script>	La salida muestra el texto escapado <script>... sin ejecutar el código	PHP y PERL
8	Intento XSS en mensaje	mensaje=	La salida muestra la etiqueta como texto, sin ejecutar el evento	PHP y PERL
9	Datos completamente válidos	Todos los campos con valores correctos	Resumen de los datos válidos sin errores	PHP y PERL
10	Nombre con caracteres especiales válidos	nombre=Maria Jose Munoz-Lopez	Datos aceptados y mostrados correctamente	PHP y PERL

Captura obligatoria para el informe

Para los casos 7 y 8 (XSS), mostrar en el informe tanto la captura del resultado en pantalla (texto escapado) como el código fuente HTML generado (clic derecho → Ver código fuente) que confirma que los caracteres están codificados como entidades HTML y no como etiquetas ejecutables.

6 Tabla comparativa PHP vs. PERL/CGI

Completar la siguiente tabla en el informe con los valores observados durante la practica:

Criterio	PHP 8.2	PERL 5.38 con CGI.pm
Integracion con Apache	Modulo nativo (mod_php): el interprete vive dentro del proceso de Apache. Mayor velocidad al no crear un proceso nuevo por peticion.	CGI clasico: Apache lanza un proceso Perl nuevo por cada peticion. Mayor overhead, menor rendimiento bajo carga.
Leer datos del formulario	<code>filter_input(INPUT_POST, 'campo', FILTRO)</code> o <code>\$_POST['campo']</code>	<code>\$cgi->param('campo')</code> mediante objeto <code>CGI->new</code>
Funcion de saneamiento XSS	<code>htmlspecialchars(\$val, ENT_QUOTES, 'UTF-8')</code>	<code>CGI::escapeHTML(\$val)</code>
Validacion de email	<code>filter_input(..., FILTER_VALIDATE_EMAIL)</code> — filtro integrado en el nucleo de PHP	Expresion regular manual: <code>/^[...]+\.[...]+\.[a-zA-Z]{2,}\$</code>
Validacion de entero	<code>FILTER_VALIDATE_INT</code> con opciones de rango	Expresion regular <code>/\d+/</code> + comparacion numerica
Cabecera HTTP	No es obligatorio emitirla manualmente (Apache la gestiona); <code>header()</code> para redirecciones	Obligatorio como primera salida: <code>\$cgi->header()</code> antes de cualquier <code>print</code>
Modo de tipos estrictos	<code>declare(strict_types=1)</code> en PHP 8	<code>use strict;</code> y <code>use warnings;</code>
Salida HTML	Intercalado PHP/HTML con bloques <code><?php></code> o con <code>print</code>	<code>print</code> , heredoc o <code>CGI->start_html()</code>
Permisos de archivo	No requiere permisos especiales en Windows/XAMPP	chmod 755 obligatorio en Linux/Mac (<code>cgi-bin</code>)
Curva de aprendizaje	Media: sintaxis intuitiva, documentacion extensa en espanol	Alta: sintaxis expresiva pero densa; muchas formas de hacer lo mismo
Casos de uso actuales	Desarrollo web dinamico, CMS (WordPress, Drupal), APIs REST	Administracion de sistemas, bioinformatica, automatizacion de texto, scripting

7 Preparacion del documento PDF a entregar

El PDF debe incluir las siguientes secciones en el orden indicado. Extensión recomendada: 3–5 paginas.

1. **Portada:** nombre del estudiante, codigo, asignatura, semana y fecha.
2. **Capturas de instalacion:** salida de `php -v` y `perl -v` en terminal, y captura del panel de XAMPP con Apache activo.
3. **Captura del formulario HTML** tal como se visualiza en el navegador.
4. **Evidencia de los 10 casos de prueba:** una captura por cada caso (puede reducirse a 5 si el docente lo indica), mostrando claramente el dato enviado y la respuesta del servidor, tanto en

PHP como en PERL. Para los casos XSS, incluir además el código fuente HTML.

5. **Tabla comparativa** completada con observaciones propias del estudiante (no copiar textualmente la tabla de esta guía; añadirle una columna “Observación personal”).
6. **Reflexión final** (mínimo 5 líneas): ¿qué diferencia le pareció más significativa entre PHP y PERL? ¿Bajo qué condiciones elegiría uno u otro para un proyecto real?

Criterios de revisión del docente

El docente verificará: (a) que ambos scripts funcionan correctamente ante los 10 casos de prueba; (b) que el saneamiento XSS es efectivo (código fuente del HTML revisado); (c) que no hay errores de PHP/PERL en el log de Apache; (d) que la reflexión es propia y no generada automáticamente.

Referencia rápida de errores comunes

Error	PHP	PERL/CGI
Error 500 Internal Server	Revisar el log de Apache: <code>xampp/apache/logs/error.log</code>	El script PERL tiene error de sintaxis, falta el shebang, o falta <code>chmod 755</code>
Página en blanco	Activar <code>display_errors=On</code> en <code>php.ini</code>	Asegurarse de que <code>CGI::Carp</code> <code>qw(fatalToBrowser)</code> está cargado
Cabecera ya enviada	No imprimir nada antes de <code>header()</code> ni antes del bloque PHP	No hacer <code>print</code> de nada antes de <code>\$cgi->header()</code>
No se instala CGI.pm	—	Ejecutar: <code>cpan install CGI</code> o <code>perl -MCPAN -e 'install CGI'</code>
<code>filter_input</code> devuelve null	Verificar que <code>extension=filter</code> está activa en <code>php.ini</code>	—

Bibliografía de apoyo

1. Nixon, R. (2021). *Learning PHP, MySQL & JavaScript* (6th ed.). O'Reilly Media. ISBN: 978-1-492-06229-9
2. Schwartz, R. L., foy, b. d., & Phoenix, T. (2021). *Learning Perl: Making easy things easy and hard things possible* (8th ed.). O'Reilly Media. ISBN: 978-1-492-09495-1
3. Welling, L., & Thomson, L. (2016). *PHP and MySQL web development* (5th ed.). Addison-

Wesley. ISBN: 978-0-321-83389-1

4. OWASP Foundation. (2021). *OWASP Top 10: 2021*. <https://owasp.org/Top10/>
5. PHP Group. (2024). *PHP Manual: filter_input*. <https://www.php.net/manual/es/function.filter-input.php>
6. Perl Foundation. (2024). *CGI — Simple Common Gateway Interface class*. <https://metacpan.org/pod/CGI>